OCL Constraints

1. User

```
context User
inv EmailNotEmpty:
    self.email <> ''

inv EmailValidFormat:
    self.email.matches('[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}')

inv PasswordNotEmpty:
    self.password <> ''

inv PasswordMinLength:
    self.password.size() >= 6

inv FirstNameValid:
    self.firstName <> '' and self.firstName.size() >= 3

inv LastNameValid:
    self.lastName <> '' and self.lastName.size() >= 3

inv EmailUnique:
    User.allInstances()->isUnique(email)
```

---

2. Project

```
context Project
inv NameNotEmpty:
    self.name <> '' and self.name.size() >= 3

inv ProjectNameUnique:
    Project.allInstances()->isUnique(name)

inv DefaultVersion:
    self.currentVersion = null implies self.currentVersion = '0.0.0'

inv ValidVersionFormat:
    self.currentVersion.matches('[0-9]+\.[0-9]+\.[0-9]+')

inv OwnerExists:
    self.owner <> null
```

3. Invitation

```
context Invitation
inv ValidState:
   Set{'Pending','Expired','Accepted','Canceled'}->includes(self.state)

inv SenderExists:
   self.sender <> null

inv ReceiverExists:
   self.receiver <> null

inv ProjectExists:
   self.projectInvitor <> null

inv RoleValid:
   Set{'Leader','Tester','Reviewer','Developer'}->includes(self.role)

inv LinkNotEmpty:
   self.invitationLink <> ''

inv LinkUnique:
   Invitation.allInstances()->isUnique(invitationLink)
```

◇ prevent duplicate invitation

```
context Invitation
inv NoDuplicateInvitation:
   Invitation.allInstances()
   ->select(i |
      i.receiver.email = self.receiver.email and
      i.projectInvitor = self.projectInvitor and
      i <> self
   )
   ->forAll(i |
      i.state = 'Expired' or i.state = 'Canceled'
   )
```

◇ 13 day condition

```
context Invitation
inv ResendAfter13Days:
```

```
    self.state = 'Pending' implies
    (Date::now() - self.createdAt) <= 13
```

---

4. Member (Superclass)

```
context Member
inv MustHaveUser:
    self.user <> null

inv MustHaveProject:
    self.project <> null
```

---

5. Owner

```
context Owner
inv UniqueOwnerPerProject:
    Owner.allInstances()
    ->select(o | o.project = self.project)
    ->size() = 1
```

---

6. Leader

```
context Leader
inv CanBypassPipeline:
    self.canBypass = true
```

---

7. Developer

```
context Developer
inv CanCreateUpdate:
    self.updates->size() >= 0
```

---

8. Tester

```
context Tester
inv CanCreateTests:
    self.testCases->size() >= 0
```

---

9. Reviewer

```
context Reviewer
inv CanReviewUpdates:
    self.codeReviews->size() >= 0
```

---

10. Pipeline

```
context Pipeline
inv ValidPhases:
    Set{'Modify','Testing','Review','Merged','Rejected'}->includes(self.currentPhase)
```

---

11. File (Code)

```
context File
inv FileNotEmpty:
    self.name <> ''

inv ValidExtension:
    Set{'.java','.py','.js','.cpp','.c','.html','.css','.json','.xml'}
    ->exists(ext | self.name.endsWith(ext))
```

---

12. Task

```
context Task
inv TitleNotEmpty:
    self.title <> ''
```

```
inv StatusValid:
   Set{'Open','InProgress','Closed'}->includes(self.status)

inv PriorityValid:
   Set{'Low','Medium','High'}->includes(self.priority)
```

---

13. Bug

```
context Bug
inv DescriptionNotEmpty:
   self.description <> ''

inv StatusValid:
   Set{'Open','Fixed','Closed'}->includes(self.status)

inv SeverityValid:
   Set{'Low','Medium','High','Critical'}->includes(self.severity)
```

---

14. Update

```
context Update
inv TitleNotEmpty:
   self.title <> ''

inv MustHaveDeveloper:
   self.developer <> null

inv ValidStatus:
   Set{
   'Under Testing',
   'Under Review',
   'Merged',
   'Rejected',
   'Modify'
   }->includes(self.status)

inv CannotEditAfterModifyPhase:
   self.status <> 'Modify' implies self.isEditable = false
```

---

15. CodeReview

```
context CodeReview
inv ReviewerExists:
   self.reviewer <> null

inv UpdateExists:
   self.update <> null

inv StatusValid:
   Set{'Approved','Rejected','NeedsChanges'}->includes(self.status)
```

---

16. TestCase

```
context TestCase
inv NameNotEmpty:
   self.name <> ''

inv MustHaveAtLeastOneScenario:
   self.successScenarios->size() >= 1

inv ResultValid:
   Set{'Pass','Fail','NotSet'}->includes(self.result)
```

Additional (Optional) Constraints

---

1. TestReport Class

Based on your description:

- testCases : List<TestCase>
- feedBack : String

Constraints

```
context TestReport
inv MustHaveAtLeastOneTestCase:
   self.testCases->size() >= 1

inv FeedbackNotEmpty:
   self.feedBack <> ''

inv AllTestCasesBelongToSameUpdate:
   self.testCases->forAll(t | t.update = self.update)
```

---

2. Prevent Editing TestReport Outside Testing Phase

```
context TestReport
inv EditOnlyInTestingPhase:
   self.update.status <> 'Testing' implies self.isEditable = false
```

Meaning:
If the associated update is **not in the Testing phase**, the TestReport must not be editable.

---

3. TestCase Constraints (Extended)

```
context TestCase
inv MustBelongToReport:
   self.testReport <> null

inv ResultMustBeSetBeforeSubmit:
   self.testReport.isSubmitted = true implies
   self.result <> 'NotSet'
```

---

4. Prevent Editing CodeReview Outside Review Phase

```
context CodeReview
inv EditOnlyInReviewPhase:
```

```
self.update.status <> 'Review' implies self.isEditable = false
```

Meaning:
The CodeReview can only be edited during the **Review phase**. Otherwise, it must be locked.

---

5. Additional Important Constraints

Prevent Submitting TestReport if Any TestCase Has No Result

```
context TestReport
inv AllTestsMustHaveResultBeforeSubmit:
  self.isSubmitted = true implies
  self.testCases->forAll(t | t.result <> 'NotSet')
```

Update Status Based on Test Results

```
context Update
inv FailedTestReturnModify:
  self.testReport.testCases->exists(t | t.result = 'Fail')
  implies self.status = 'Modify'
```

If All Tests Pass → Move to Review Phase

```
context Update
inv AllPassedGoToReview:
  self.testReport.testCases->forAll(t | t.result = 'Pass')
  implies self.status = 'Review'
```

---

6. Important Consistency Constraint

```
context TestReport
inv CannotModifyAfterSubmit:
  self.isSubmitted = true implies self.isEditable = false
```

---

7. Prevent modifying code outside **Modify phase**

```
context Update
inv EditOnlyInModify:
   self.status <> 'Modify' implies self.codeLocked = true
```

---

8. All tests must pass before review

```
context Update

inv AllTestsPassedBeforeReview:
   self.status = 'Review' implies
   self.testCases->forAll(t | t.result = 'Pass')
```

---

9. If a test fails, it should be returned for modification

```
context Update

inv FailedTestReturnModify:
   self.testCases->exists(t | t.result = 'Fail')
   implies self.status = 'Modify'
```

---

10. At least one test case is required

```
context Update
inv AtLeastOneTest:
   self.testCases->size() >= 1
```